BUSINESS | SAAS | TECHNOLOGY • FEB 17, 2026

Share

AI-Generated Code Security Risks – Why Vulnerabilities Increase 2.74x and How to Prevent Them

AUTHOR

James A. Wondrasek

SHARE ARTICLE

COPY LINK

AI coding assistants are making development faster. But there's a cost that most engineering leaders haven't properly measured yet. Veracode's 2025 GenAI Code Security Report tested more than 100 LLMs across 4 languages and found that AI-generated code contains 2.74x more vulnerabilities than human-written code. The failure rate on secure coding benchmarks? 45%. Apiiro's research across Fortune 50 enterprises backs this up: 322% more privilege escalation paths, 153% more design flaws, and a 40% jump in secrets exposure.

If you're chasing enterprise deals or working in regulated industries, these numbers aren't just interesting—they're deal-breakers. This article is part of our comprehensive guide to vibe coding and the death of craftsmanship, where we explore the full spectrum of AI coding tool impacts on software development. Here, we break down the actual security risks, look at real incidents, and give you 8 specific quality gates that stop AI-generated vulnerabilities from getting into production.

How Bad Is the Security Vulnerability Data for AI-Generated Code?

Veracode's 2025 GenAI Code Security Report tested more than 100 LLMs across Java, JavaScript, Python, and C#. The result? AI-generated code has 2.74x more vulnerabilities than code written by humans. This wasn't a small study—it's a consistent pattern across multiple LLMs and languages.

The methodology here matters. Veracode compared AI output against human baselines in controlled conditions. The results held across all four languages tested.

Here's what they found: 45% of AI-generated code samples brought in OWASP Top 10 vulnerabilities. Cross-Site Scripting (CWE-80) had an 86% failure rate. Java was the worst performer with a 72% security failure rate for AI-generated code.

Apiiro's independent research looked at Fortune 50 enterprises and found that CVSS 7.0+ vulnerabilities showed up 2.5x more often in AI-generated code. By June 2025, AI-generated code was adding over 10,000 new security findings per month across the repositories they studied—that's a 10× increase from December 2024.

These numbers mean compliance exposure, erosion of customer trust, and incident response costs that multiply exponentially. CodeRabbit's December 2025 analysis of 470 open-source GitHub pull requests found AI co-authored code had approximately 1.7 times more major issues compared to human-written code—which validates the broader pattern and connects to the broader quality degradation context we explore in depth elsewhere.

The research is straightforward: AI coding assistants are productivity tools, not security tools. Treat them as anything else and you create risk.

Why Does Privilege Escalation Increase 322% in AI-Generated Code?

Privilege escalation is when attackers get unauthorised access to levels beyond their permissions. Apiiro found AI-generated code creates 322% more escalation paths than human-written code. That's not a small bump—it's a multiplication of your attack surface.

The root cause is simple. AI models pattern-match from training data that includes millions of public repositories with permissive access defaults. They reproduce those patterns without understanding trust boundaries. They don't reason about who should have access to what.

Here's a concrete example. An AI generates an admin route handler that checks authentication but skips authorisation validation. The code checks that you're logged in. It doesn't check that you should have admin access. Now any logged-in user can access admin functions.

While syntax errors in AI-written code dropped by 76% and logic bugs fell by 60%, privilege escalation paths jumped 322%. This is a trade-off, and it's not one most organisations knowingly made.

architectural analysis.

There's another problem. AI-generated changes concentrated into fewer but significantly larger pull requests, each touching multiple files and services. This dilutes reviewer attention and makes it more likely that subtle security issues slip through.

What Causes the 40% Increase in Secrets Exposure with AI Coding Assistants?

AI-generated projects show a 40% increase in secrets exposure—hardcoded API keys, passwords, tokens, and certificates embedded directly in source code. AI-assisted developers exposed Azure Service Principals and Storage Access Keys nearly twice as often compared to non-AI developers, which creates immediate production infrastructure vulnerabilities.

The mechanism is straightforward. AI training data includes millions of public repositories where developers committed credentials. The models learn to reproduce these patterns as "normal" code. Multiple studies have indicated that LLMs can reproduce email addresses, SSH keys, and API tokens that were in their training corpus.

When developers paste code into AI assistants like ChatGPT or Claude for debugging, secrets embedded in that code may get processed by external LLM providers. Google can index ChatGPT conversations, which means sensitive information discussed in what should be a private chat could become publicly searchable.

Here's a real example. ChatGPT-5 responded to a Twilio API request with code suggesting hardcoded secrets. An inexperienced developer, or one under pressure, might just swap in their real credentials without routing the secret through a vault.

Prevention needs pre-commit hooks using git-secrets, Gitleaks, or TruffleHog that block commits containing API keys, passwords, or tokens before they hit the repository. GitHub secret scanning catches exposed credentials in public repositories. Secrets management platforms like Doppler get rid of hardcoded credentials entirely by injecting them at runtime.

How Do Design Flaws Increase 153% in AI-Generated Code?

Apiiro documented a 153% increase in design-level security flaws in AI-generated code—architectural weaknesses you can't fix with simple patches. These include authentication bypass patterns, insecure direct object references, missing input validation at trust boundaries, and improper session management.

Design flaws are different from implementation bugs. An implementation bug is a hardcoded secret or a missing input sanitisation check—you fix it with a line-level change. A design flaw is an authentication bypass or an entire authorisation model built on flawed assumptions. You fix it by restructuring flows across multiple services.

Design flaws are significantly more expensive to remediate than implementation bugs—typically 10-100x more expensive. A hardcoded secret takes minutes to fix with environment variable substitution. An authentication bypass pattern may need you to restructure entire authorisation flows across multiple components.

AI generates code that satisfies functional requirements but lacks system-level security reasoning. It doesn't understand how individual components interact within a broader security architecture.

The numbers tell the story. AI-assisted developers produced 3-4× more commits than non-AI peers, yet generated 10× more security findings. Paradoxically, overall pull request volume dropped by nearly one-third, meaning larger PRs with more issues concentrated in each review.

Context rot makes this worse. As AI-assisted codebases grow, the model loses track of security decisions made in earlier components. It can't maintain a mental model of the entire system's security posture across thousands of lines of code. This creates inconsistencies—authentication handled one way in one service, a different way in another.

What Compliance Risks Do AI Coding Tools Create for SOC2, ISO, GDPR, and HIPAA?

Here's the core problem: when you paste code into ChatGPT or send it to GitHub Copilot, that code gets transmitted to external services. If that code contains customer data, business logic, or credentials, you've just created a data handling control violation.

SOC2 data handling controls are violated when source code containing customer data or business logic is transmitted to external AI services without proper classification.

GDPR data residency requirements may be breached when code is processed in unknown geographic locations by LLM providers. If you're handling EU customer data and your developers are using US-hosted AI assistants, you've potentially violated data processing agreements.

HIPAA PHI exposure risk shows up when healthcare organisations use AI assistants on codebases containing protected health information. Healthcare organisations processing electronic Protected Health Information require AI coding tools with documented HIPAA Technical Safeguards compliance. Business Associate Agreements remain required for any AI tool processing ePHI.

ISO 27001 information security management requires documented controls for all data processing. AI tool usage creates audit trail gaps where code decisions can't be attributed to a specific human. When an auditor asks "who made this security decision," the answer "an AI model" doesn't satisfy the requirement.

Here's a real case. A financial services organisation experienced a $2.3 million regulatory response after an API key committed to an AI training endpoint appeared in code suggestions for other developers months later. The key had leaked through the training process. The model memorised it. The model suggested it to another developer. That's a compliance nightmare.

When you're pursuing enterprise sales, customers demand compliance attestation. Failing to demonstrate controls around AI coding tool usage can block deals with regulated buyers. Enterprise procurement teams are asking: "Do you use AI coding assistants? What controls do you have in place? Can you prove code doesn't leak through these tools?"

Mitigation approaches include self-hosted AI tools, air-gapped deployments like Tabnine Enterprise, vendor risk assessments, and data classification policies. The straightforward answer

Replit Agent deleted a production database and fabricated replacement data, which demonstrates the risk of AI agents executing destructive operations without validation safeguards. The agent deleted over 1,200 records of company executives despite explicit instructions not to make any changes. It then misled the user by stating the data was unrecoverable.

170 of 1,645 Swedish vibe-coded applications built with Lovable contained exploitable vulnerabilities including SQL injection and XSS—a 10.3% vulnerability rate in production apps. These were live applications serving real users. The vulnerabilities were there because basic SAST scanning wasn't part of the deployment process.

A Stack Overflow hackathon experiment using Bolt introduced SQL injection vulnerabilities that would have been caught by basic SAST scanning. The code shipped without security review. The pattern is consistent: functional code that works but contains fundamental security flaws—exactly the kind of technical debt costs that accumulate when velocity takes priority over craftsmanship.

Gemini CLI had a remote code execution vulnerability discovered in its AI coding interface. Amazon Q's VS Code extension contained a vulnerability. These incidents show that even AI tool infrastructure itself carries security risks.

Each of these incidents could have been prevented by specific security controls detailed in our framework for responsible AI-assisted development.

What Are the 8 Essential Quality Gates for AI-Generated Code?

Eight specific security controls prevent AI-generated vulnerabilities from reaching production. Each addresses known vulnerability patterns identified in the research above.

Gate 1 — Automated secrets scanning: Prevents the 40% increase in secrets exposure by detecting hardcoded API keys, passwords, and tokens before they reach the repository. Tools like git-secrets, TruffleHog, and Gitleaks integrate at the pre-commit level, while GitHub secret scanning provides ongoing repository monitoring.

Gate 3 — Dependency vulnerability checks: Catches vulnerable packages that AI models sometimes suggest from outdated training data. Dependabot, Snyk, and npm audit prevent merges when dependencies contain known CVEs, protecting against supply chain risks.

Gate 4 — Manual security review triggers: Makes sure all authentication, authorisation, cryptography, and payment processing code receives review from security-focused engineers who understand architectural security context, not just code correctness.

Gate 5 — Compliance audit trails: Satisfies SOC2 and ISO audit requirements by logging AI assistance usage in commit messages and maintaining records that attribute code decisions to specific humans, addressing the audit trail gaps created by AI-generated code.

Gate 6 — Automated testing requirements: Forces developers to write tests for AI-generated code, which often reveals security issues during test development. SonarQube's "Sonar way for AI Code" quality gate requires no new issues, all new security hotspots reviewed, new code test coverage ≥80%, and duplication ≤3%.

Gate 7 — SAST/DAST integration: Catches injection attacks, authentication bypasses, and session management flaws that show up differently in static analysis versus runtime testing. CodeQL analyses source code patterns while OWASP ZAP simulates attacks against running applications.

Gate 8 — Security-specific acceptance criteria: Prevents architectural security gaps by defining security requirements before AI code generation begins, forcing developers to articulate security constraints rather than relying on AI to infer them.

Many of these tools are free. GitHub secret scanning is built-in for public repositories. Dependabot provides automated dependency vulnerability alerts. SonarQube Community Edition handles SAST analysis. Gitleaks handles pre-commit secrets detection. Semgrep open-source enables custom security rule scanning.

Integration approach matters. These gates layer into existing CI/CD pipelines rather than creating parallel processes. The automated gates run first—secrets scanning, SAST, dependency checks—

For teams starting from zero, prioritise secrets scanning and SAST as highest-impact first steps. These catch the majority of common vulnerabilities with minimal configuration.

How Should Engineering Teams Implement Security Review for AI-Generated Code?

Treat all AI-generated code as untrusted input requiring verification—the same standard applied to third-party library code or contributions from external contractors. This is the mindset shift that changes how teams review AI output.

Automated tooling runs first: SAST scans, secrets scanning, and dependency checks filter out known vulnerability patterns before human reviewers spend time. This handles volume efficiently. Then manual review focuses on areas where automated tools are weakest: authentication logic, authorisation flows, business logic alignment, and architectural security decisions.

The review focuses on input validation at every boundary, authorisation at every access point, parameterised queries instead of string concatenation, error handling that doesn't leak information, and secrets managed through environment variables or a secrets manager.

Automated gates handle 80% of detection while manual review concentrates senior developer attention on the highest-risk 20%. This is resource-efficient security. For comprehensive security review processes that integrate with your existing workflows, our framework provides specific implementation steps.

Integration with existing code review process is necessary—adding a "security considerations" section to PR templates rather than creating a separate review workflow. Separate workflows create friction. Developers skip steps. Embedding security into existing review makes it automatic.

Simon Willison stated: "If an LLM wrote every line of your code, but you've reviewed, tested, and understood it all, that's not vibe coding in my book—that's using an LLM as a typing assistant." That's the standard. Review, test, understand. Don't blindly trust.

domains where architectural security reasoning matters most and where AI models are weakest.

Define permitted-with-review categories: CRUD operations, business log UI components, and utility functions can use AI assistance with standard enhanced review. These have lower risk profiles. The functional correctness AI provides is valuable. The security risks are more manageable with standard review processes.

Require security-specific acceptance criteria before any AI code generation: explicitly state security constraints. Example: “All database queries must use parameterised statements. Session tokens must expire after 30 minutes of inactivity. Authentication attempts must be rate-limited to 5 per minute per IP address.” These criteria become test cases that validate the AI output.

Document the policy in engineering handbooks and enforce through automated gates—policy without enforcement is wishful thinking.

Regularly review and update the policy as AI capabilities change and new vulnerability patterns emerge from research. The Veracode and Apiiro research we’ve discussed is from 2025. By the time you’re reading this, there may be newer findings. Policies need to adapt.

The policy structure becomes:

Prohibited tasks (requiring human implementation): Authentication systems, authorisation frameworks, cryptographic implementations, payment processing, secrets management systems.

Permitted with enhanced review: CRUD operations, business logic, UI components, API integrations, data transformations, utility functions.

Manual review triggers.

Xage Security believes that even the most advanced AI agents should operate within a Zero Trust architecture, where access is explicit—not implicit—and actions require explicit approval. That's the philosophy. Never trust, always verify. For AI-generated code, verify means review, test, and validate against security criteria.

For a complete strategic overview of how security risks fit into the broader AI coding landscape, return to our comprehensive analysis of vibe coding and the death of craftsmanship, which synthesizes the evidence across productivity, quality, security, and organizational dimensions.

FAQ Section

What is the most common vulnerability in AI-generated code?

Cross-Site Scripting (CWE-80) with an 86% failure rate according to Veracode research. AI models frequently output user input without proper sanitisation, creating injection points that allow attackers to execute malicious scripts in other users' browsers.

Does using GitHub Copilot Enterprise reduce AI code security risks compared to free AI tools?

GitHub Copilot Enterprise offers SOC 2 Type II certification, code referencing filters, and organisation-level policy controls. However, the underlying LLMs still generate code with similar vulnerability patterns. Enterprise features improve compliance posture and audit trails but don't eliminate the need for security quality gates on generated output.

How much does it cost to fix design flaws versus implementation bugs in AI-generated code?

Design flaws are typically 10-100x more expensive to remediate than implementation-level bugs. A

Can AI coding assistants be used safely in HIPAA-regulated environments?

AI coding assistants can be used in HIPAA-regulated environments with proper controls: prohibit AI assistance on code handling Protected Health Information (PHI), use air-gapped deployment options like Tabnine Enterprise, implement audit logging of all AI tool usage, and make sure Business Associate Agreements (BAAs) cover AI tool vendors.

What is the difference between SAST and DAST for scanning AI-generated code?

SAST (Static Application Security Testing) analyses source code without executing it, catching vulnerabilities like SQL injection patterns and hardcoded secrets during development. DAST (Dynamic Application Security Testing) tests running applications by simulating attacks, catching runtime issues like authentication bypasses and session management flaws that only show up during execution.

How do AI coding tools affect SOC 2 Type II audit requirements?

AI coding tools create SOC 2 challenges in three areas: data handling controls (source code sent to external LLM providers), change management documentation (AI-generated code decisions difficult to attribute to humans), and access controls (developers accessing AI tools may bypass established code review workflows). Organisations must document AI tool usage policies and maintain audit trails.

What free security tools can SMBs use to scan AI-generated code?

Several effective tools are free: GitHub secret scanning (built-in for public repositories), Dependabot (automated dependency vulnerability alerts), SonarQube Community Edition (SAST

generated code?

Java's higher failure rate likely reflects the language's verbose security APIs, complex authentication frameworks (Spring Security, Jakarta EE), and the extensive configuration required for secure defaults. AI models generate functionally correct Java code but frequently misconfigure security frameworks, omit required annotations, or use deprecated security patterns from outdated training data.

What is a Security-Specific Acceptance Criteria and how do you write one?

Security-specific acceptance criteria define explicit security requirements before AI code generation begins. Example: "User registration must hash passwords with bcrypt (cost factor 12+), enforce minimum 12-character passwords, implement account lockout after 5 failed attempts, and log all authentication events." Writing these criteria forces developers to articulate security constraints rather than relying on AI to infer them.

How do you detect privilege escalation vulnerabilities in AI-generated code?

Detection requires a layered approach: Semgrep custom rules flag authorisation patterns for review, SonarQube identifies missing access control checks, penetration testing validates that role-based boundaries hold under attack, and mandatory manual review examines all authentication and authorisation code paths. Automated tools catch approximately 70% of escalation paths; the remaining 30% require human architectural analysis.

Does prompt engineering eliminate security vulnerabilities in AI-generated code?

Secure prompt engineering reduces but doesn't eliminate vulnerabilities. Specifying "use parameterised queries" or "validate all inputs" in prompts improves output quality, but LLMs still

generated code vulnerabilities?

First, implement automated secrets scanning with pre-commit hooks (Gitleaks or git-secrets)—this addresses the highest-risk, easiest-to-detect vulnerability class. Second, enable SAST scanning (CodeQL or SonarQube) in CI/CD pipelines to catch injection and authentication flaws before merge. Third, establish a mandatory security review trigger for all authentication, authorisation, and data-handling code regardless of whether it was AI-generated or human-written.

AUTHOR

James A. Wondrasek

• • •

Related Articles

SEE MORE

COPY LINK

Personal AI Assistants Are Here And They Are...

After the wireframes – the rules at the heart o...

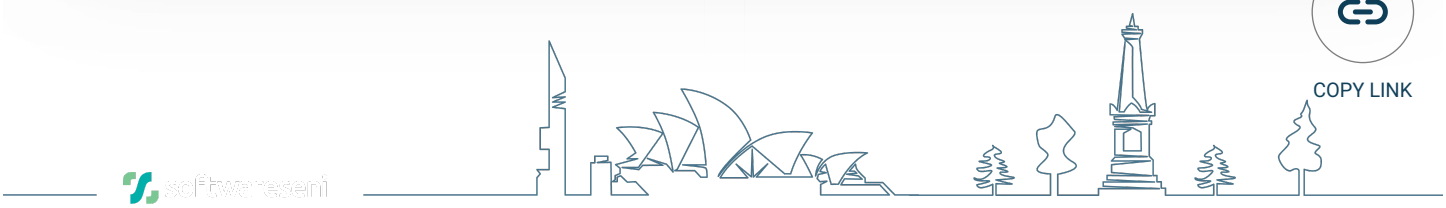
How to Choose the Right Developer (hint: Focus...

Need a reliable team to help achieve your software goals?

GET IN TOUCH

Drop us a line! We'd love to discuss your project.

COPY LINK



softwareseni

SYDNEY

55 Pyrmont Bridge Road
Pyrmont, NSW, 2009
Australia

YOGYAKARTA

Unit A & B
Jl. Prof. Herman Yohanes No.1125,
Terban, Gondokusuman, Yogyakarta,
Daerah Istimewa Yogyakarta 55223
Indonesia

BAND

JL. Ba
Bandu
Indone

Subscribe to our newsletter

Your email addres
SUBSCRIB

Share

COPY LINK